

SplitDude

Complete Technical Handbook
System Design & Architecture Guide
FAANG Interview Preparation Manual

Version 1.0.0

Author: Sairam Parasa

Released: July 2026

Table of Contents

- Chapter 1: Project Overview
- Chapter 2: Project Architecture
- Chapter 3: Complete Tech Stack
- Chapter 4: Database Schema & Design
- Chapter 5: Authentication & Session Management
- Chapter 6: AWS Cloud Services & Integrations
- Chapter 7: React & UI Architecture
- Chapter 8: Performance Optimizations
- Chapter 9: Security & Threat Mitigation
- Chapter 10: Features Walkthrough
- Chapter 11: System Design Deep Dive
- Chapter 12: Architectural Decisions & Rationale
- Chapter 13: Interview Preparation (150+ Questions)
- Chapter 14: Possible Cross Questions
- Chapter 15: Project Demo Walkthrough
- Chapter 16: Resume Pitch Scripts
- Chapter 17: Common Mistakes to Avoid
- Chapter 18: FAANG Level Architecture Assessment
- Chapter 19: Quick Revision Cheat Sheets

Chapter 1: Project Overview

1.1 What is SplitDude?

SplitDude is a cloud-native, serverless expense collaboration platform built around privacy-first identity (no phone numbers), lightning-fast expense splits, modern glassmorphic UI, and deep AWS cloud integrations. It is designed to solve the common friction points of shared group financial transactions by enabling users to split expenses, track debts, and log settlements in under 30 seconds.

1.2 Problem Statement & Motivation

Traditional expense splitting applications often require users to share sensitive personal details like phone numbers or email addresses, introducing privacy concerns. Furthermore, they suffer from complex transaction volumes where multiple members owe each other cross-debts. SplitDude addresses this by utilizing a custom unique Split ID system (e.g., SPD7H4K2M) for complete anonymity, and an advanced greedy debt simplification algorithm to reduce the total transaction count to a minimum.

1.3 Real-World Use Cases

1. Shared Apartment Expenses: Flatmates splitting rent, utilities, internet, groceries, and household goods. 2. Travel & Trips: Group of friends travelling together and recording shared expenses like flights, hotels, dinners, and taxi rides. 3. Event Planning: Splitting costs for planning parties, dinners, or outings where multiple members pay for different items. 4. Daily Ledgers: Recording mutual debts among friends, classmates, or colleagues over time.

1.4 Target Audience

SplitDude is targeted towards privacy-conscious users, flatmates, travel groups, event planners, and teams who want a zero-friction, premium, and highly visual dashboard to manage group finances without sharing personal phone numbers or contact details.

1.5 Key Features

- Privacy-First Profile Creation: Direct Supabase auth sync that generates a unique code for peer connection.
- Interactive Command Palette: Global shortcut (Ctrl+K) to search and jump to groups, friends, or settings.
- Intelligent AWS Textract OCR: Auto-extracts total bill amount, vendor names, and billing dates from receipt files.
- Equal, Exact, & Percentage Splitting: Highly flexible ledger modes to divide bills precisely.
- Greedy Debt Simplification: Solves multi-party debts using a max-heap greedy algorithm.

1.6 Future Scope & Product Vision

The product vision is to become the go-to open-source ledger application for serverless multi-currency group accounting. Future improvements include: integration of payment gateways (UPI, Venmo), multi-currency automatic conversions, receipt line-item splitting, and native push notifications via web sockets.

Chapter 2: Project Architecture

2.1 Architecture Overview

SplitDude utilizes a modern serverless architecture combining Next.js (App Router), Supabase (PostgreSQL with Row-Level Security), and AWS Serverless services (API Gateway, Lambda, S3, and Textract).

2.2 Frontend & Next.js Router

The frontend is built using Next.js 16 (App Router) and React 19. It uses server components for secure page rendering and client components for responsive widgets, modal triggers, and animations. The page routing is isolated under (auth) and (dashboard) layouts to manage public onboarding and authenticated dashboard states.

2.3 Supabase Database Integration

Supabase acts as our PostgreSQL hosting provider. The Next.js client connects directly to Supabase using anonymized keys. Postgres Row-Level Security (RLS) is enabled on all tables to ensure that users can only read or write rows that belong to them or their group memberships.

2.4 AWS OCR Integration Flow

To scan a receipt, the client uploads the file to the Next.js API route (/api/receipts/analyze). The route base64-encodes the file and forwards it to AWS API Gateway. API Gateway triggers a Python Lambda function. The Lambda uploads the file to an Amazon S3 temporary bucket (auto-expired after 1 day) and invokes Amazon Textract AnalyzeExpense API. The parsed merchant, date, and amount are returned back to Next.js to auto-fill the form.

2.5 System Flow Diagrams

Please refer to docs/Architecture.md for detailed sequence flow diagrams of all routes.

Chapter 3: Complete Tech Stack

3.1 Core Frameworks & Languages

- Next.js 16: Selected for server-side rendering, routing structure, and backend serverless route handlers. Alternatives like Vite + Express were rejected to avoid managing a separate server instance.
- React 19: Leveraged for the virtual DOM, hook-based components, and state management. React 19 compiler warnings were fixed to support strict builds.
- TypeScript 5: Guarantees compile-time type safety across database responses and component properties.

3.2 Styling & Presentation Libraries

- Tailwind CSS: Selected for utility-first styling to build a premium glassmorphic UI. Custom CSS variables provide seamless themes.
- Framer Motion 12: Used for hardware-accelerated fluid transitions (modals, sheet sliders, graphs).
- Recharts 3: Used for responsive financial analysis widgets (pie charts, line charts, bar ledgers).
- Lucide React: Providing modern, consistent outline icons.

3.3 Data Fetching & State Caching

- TanStack React Query 5: Selected as the core state caching provider. It manages caching, background refetching, and cache invalidation on database mutations. Redux was rejected as it introduces unnecessary boilerplate for database-driven state.

Chapter 4: Database Schema & Design

4.1 Database Overview

SplitDude uses Supabase PostgreSQL. Below is the comprehensive breakdown of the database schema, tables, relations, and index choices.

4.2 Profiles Table

Stores user details. Primary key is the user UUID referencing auth.users. Contains a unique Split ID (unique_code) which is indexed for quick peer searches.

4.3 Friends & Group Members Table

- friends: Maps user_id_1 and user_id_2 representing friendships. Indexed on both columns to optimize friends lists queries.
- group_members: Connects user_id and group_id. Enforces unique constraints so a user can join a group only once.

4.4 Expenses & Splits Table

- expenses: Contains amount, title, paid_by, and split_mode (equal, exact, percentage).
- expense_splits: Stores the split share for each group member referencing the expense. Enforces check constraints to ensure amounts sum to the total expense.

Chapter 5: Authentication & Session Management

5.1 Supabase Auth

SplitDude implements email/password authentication using Supabase Auth. It generates a secure JWT token on successful login.

5.2 JWT Cookie Integration

To secure Next.js API routes and server components, the Supabase SSR package (`@supabase/ssr`) sets the JWT token as an HTTP-only cookie. This ensures the session persists across pages and is automatically sent to Next.js route handlers.

5.3 Middleware Protection

The Next.js middleware interceptor (`/src/middleware.ts`) inspects the session cookie on every request. If a user attempts to access dashboard routes (`/home`, `/groups`) without a valid token, the middleware redirects them to `/login`.

Chapter 6: AWS Cloud Services & Integrations

6.1 AWS Lambda (Python 3.12)

The receipt processor runs inside a serverless Lambda function. It decodes base64 receipt files from Next.js, saves them temporarily to S3, and calls Amazon Textract AnalyzeExpense.

6.2 Amazon S3 Storage

Used as a temporary staging bucket. S3 lifecycle policy is set to auto-delete objects after 24 hours to prevent storage accumulation and safeguard data privacy.

6.3 AWS API Gateway

Exposes the Lambda function as a secure HTTP API. Next.js proxies all OCR requests through the API Gateway, hiding the invoke URL from the client browser.

6.4 Amazon Textract

An AI/ML service that extracts structured vendor names, total billing figures, and invoice dates. It eliminates the need to build and maintain custom OCR training models.

Chapter 7: React & UI Architecture

7.1 Atomic Component Structure

The UI is structured into reusable, atomic layouts, including form modals (ExpenseModal, SettleModal), navigations (Sidebar, Header), and widgets (BalanceCard, SettlementList).

7.2 Custom Hooks

We use custom React hooks to isolate state logic from the presentation layer. For example, local autocomplete for member adding uses useEffect hooks tied to the friends list query.

7.3 State Management

Unified client state is managed via React Query cache, eliminating manual Context propagation or prop drilling for database states.

Chapter 8: Performance Optimizations

8.1 Caching Strategy

TanStack React Query handles caching. Fetch states are query-cached with keys like ['groupDetail', groupId]. Mutations invalidate these keys, triggering lazy refetching only when required.

8.2 Real-Time Autocomplete Filtering

Rather than calling the Supabase profiles table on every keystroke when adding friends to a group, the search logic filters the pre-fetched friends list in memory. This reduces network request counts to zero.

8.3 Code Splitting & Dynamic Imports

Heavy libraries like Recharts are lazy-loaded on demand using Next.js dynamic imports, reducing the initial JS bundle size and boosting page load speed.

Chapter 9: Security & Threat Mitigation

9.1 Postgres Row-Level Security (RLS)

RLS guarantees data isolation. A user can only access expense or settlement records if their `auth.uid()` is matched inside the `group_members` table for that group.

9.2 API Gateway CORS Policies

AWS API Gateway is locked down using CORS policies that restrict invocations to our specific Vercel production domains, preventing third-party script invocations.

9.3 Secrets Safety

S3 keys and Supabase service keys are stored inside environment variables. They are never compiled into the client-side code, keeping them safe from exposure.

Chapter 10: Features Walkthrough

10.1 Command Palette (Ctrl+K)

SplitDude features a keyboard-friendly search pill. Pressing Ctrl+K triggers a command dialog to search active groups, navigate between pages, or log out.

10.2 Equal, Exact, & Percentage Bill Splits

Supports splitting bills equally, by exact dollar amounts, or by percentage ratios. Form inputs validate that the shares sum to the total bill before allowing submission.

10.3 In-Memory Friend Search Autocomplete

Autocomplete displays matching friends as the user types. Split ID patterns automatically switch searches to DB lookups, ensuring high usability.

Chapter 11: System Design Deep Dive

11.1 High-Level Design

SplitDude operates as a distributed system with a stateless frontend (Next.js), serverless compute (AWS Lambda), and a relational database managed service (Supabase PostgreSQL).

11.2 Handling High Write Volumes

To scale for massive write volumes (e.g., millions of expenses logged simultaneously), transactions are queued. Supabase uses Postgres connection pooling (PgBouncer) to handle spikes in connections.

11.3 Caching & Read Optimization

Frequently read tables like profiles and friends are cached at the edge using Redis or browser memory caches. Database indexes on `group_id` and `user_id` optimize query times.

Chapter 12: Architectural Decisions & Rationale

12.1 Serverless vs Server-Based

Choosing AWS Lambda + API Gateway over an Express node server eliminates idle server costs, scales automatically from zero to thousands of requests, and requires zero infrastructure maintenance.

12.2 SQL (PostgreSQL) vs NoSQL (MongoDB)

Relational database features like ACID transactions, foreign keys, and unique constraints are critical for financial ledger accuracy. NoSQL databases like MongoDB lack native RLS enforcement and relationship integrity.

12.3 Next.js Server Actions vs REST endpoints

Next.js route handlers act as a backend API. This maintains a clean boundary, making it easy to create mobile app clients in the future that hit the same endpoints.

Chapter 13: Interview Preparation (150+ Questions)

13.1 Technical Q&A; Bank

Q1. What is the core functionality of SplitDude?

Expected Answer: SplitDude is a serverless expense collaboration platform that allows users to split expenses, track debts, and log settlements.

Detailed Explanation: It uses a custom unique Split ID system for anonymity, and an advanced greedy debt simplification algorithm to reduce the total transaction count to a minimum.

Follow-up Questions: How does the unique Split ID system prevent duplicate user registration?

Q2. Why did you choose Next.js instead of Create React App + Express?

Expected Answer: Next.js provides Server-Side Rendering (SSR), App Routing, and API Route Handlers in a single repository, simplifying development and deployment.

Detailed Explanation: By using Next.js route handlers, we eliminate the need to run, configure, and maintain a separate Express node server, reducing hosting costs and deployment complexity.

Follow-up Questions: How does Next.js SSR improve initial page load performance?

Q3. What is the Debt Simplification Algorithm used in SplitDude?

Expected Answer: SplitDude uses a greedy heap-based algorithm to minimize the total number of transactions required to settle all debts in a group.

Detailed Explanation: The algorithm computes net balances, separates creditors and debtors, sorts them, and matching them greedily by transferring balances from the largest debtor to the largest creditor.

Follow-up Questions: What is the time complexity of the greedy simplification algorithm?

Q4. Why did you choose Supabase over Firebase?

Expected Answer: Supabase provides a relational PostgreSQL database with Row-Level Security (RLS) and SQL schemas, whereas Firebase uses NoSQL JSON documents.

Detailed Explanation: *Financial ledgers require relation constraints, transaction rollbacks, and foreign keys. Firebase lacks direct RLS support for multi-party relationships.*

Follow-up Questions: How does Supabase handle real-time database subscriptions?

Q5. How does Row-Level Security (RLS) work in SplitDude?

Expected Answer: RLS is enabled in Supabase to restrict access to rows based on the authenticated user's ID matching the group_members table.

Detailed Explanation: This ensures that a user can only query, insert, or update rows that belong to them or their groups, preventing unauthorized data leaks.

Follow-up Questions: What is the performance overhead of enabling RLS on tables with millions of records?

Q6. How does the AWS receipt OCR pipeline work?

Expected Answer: The Next.js route base64-encodes the receipt file and sends it to AWS API Gateway, which triggers a Python Lambda function to run Textract OCR.

Detailed Explanation: *The Lambda function decodes the payload, uploads it to S3, calls Amazon Textract AnalyzeExpense, parses the total amount/vendor/date, and returns it to the client.*

Follow-up Questions: How do you secure the API Gateway invoke URL from public exposure?

Q7. Why did you use AWS Lambda instead of running OCR locally?

Expected Answer: AWS Lambda is serverless and executes OCR tasks on demand, eliminating server idling costs and auto-scaling to handle load spikes.

Detailed Explanation: Running heavy OCR processing (like Tesseract) locally on the Next.js server would block the event loop and degrade web app performance.

Follow-up Questions: What is the maximum execution duration (timeout) allowed for AWS Lambda?

Q8. How does Amazon Textract AnalyzeExpense differ from basic OCR?

Expected Answer: AnalyzeExpense uses machine learning models to detect key-value pairs (merchant name, date, total amount) instead of just raw text blocks.

Detailed Explanation: Basic OCR like Tesseract only returns raw bounding boxes and text, requiring complex regex algorithms to locate totals and vendors.

Follow-up Questions: What languages does Amazon Textract support?

Q9. How do you implement real-time autocomplete search for adding members?

Expected Answer: The autocomplete logic filters the pre-fetched friends list in browser memory, avoiding calling the database on every keystroke.

Detailed Explanation: *If the input matches a Split ID pattern (SPDXXXXXX), it switches to a single database profiles lookup once 9 characters are reached.*

Follow-up Questions: How do you handle keyboard navigation inside the autocomplete results dropdown?

Q10. Why is TanStack React Query used for state caching?

Expected Answer: It manages caching, automatic refetching, and cache invalidation on database mutations, keeping frontend data synchronized with Supabase.

Detailed Explanation: *This eliminates the need to write complex Redux actions or dispatch global Context updates to trigger re-renders after adding expenses.*

Follow-up Questions: How do you configure custom cache times and stale times in React Query?

Q11. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q12. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q13. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q14. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q15. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q16. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q17. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q18. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q19. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q20. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q21. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q22. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q23. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q24. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q25. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q26. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q27. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q28. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q29. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q30. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q31. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q32. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q33. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q34. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q35. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q36. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q37. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q38. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q39. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q40. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q41. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q42. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q43. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q44. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q45. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q46. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q47. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q48. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q49. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q50. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q51. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

***Detailed Explanation:** Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.*

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q52. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q53. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q54. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q55. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q56. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q57. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q58. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q59. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q60. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q61. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q62. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

***Detailed Explanation:** Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.*

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q63. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q64. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q65. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q66. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q67. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q68. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q69. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q70. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q71. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q72. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q73. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q74. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q75. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q76. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q77. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q78. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q79. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q80. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q81. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q82. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q83. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q84. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q85. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q86. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q87. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q88. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q89. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q90. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q91. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q92. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q93. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q94. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q95. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

***Detailed Explanation:** Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.*

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q96. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q97. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q98. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q99. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q100. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q101. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q102. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q103. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q104. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q105. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q106. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q107. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q108. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q109. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q110. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q111. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q112. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q113. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q114. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q115. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q116. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q117. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q118. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q119. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q120. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q121. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q122. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q123. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q124. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q125. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q126. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q127. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q128. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q129. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q130. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q131. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q132. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q133. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q134. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q135. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q136. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q137. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q138. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q139. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q140. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q141. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q142. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q143. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q144. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

Q145. Explain the role of TypeScript (Interfaces, Types, Enums, Generics, Type Guarding) in a production system like SplitDude.

Expected Answer: In SplitDude, TypeScript is critical because it ensures robust handling of Interfaces, Types, Enums, Generics, Type Guarding. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging TypeScript mitigates common failure cases. For example, Interfaces, Types, Enums, Generics, Type Guarding is optimized using best practices.

Follow-up Questions: How would you optimize TypeScript as the user base grows by 100x?

Q146. Explain the role of Supabase (Row-Level Security, Database Triggers, PgBouncer, Real-time Channels) in a production system like SplitDude.

Expected Answer: In SplitDude, Supabase is critical because it ensures robust handling of Row-Level Security, Database Triggers, PgBouncer, Real-time Channels. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Supabase mitigates common failure cases. For example, Row-Level Security, Database Triggers, PgBouncer, Real-time Channels is optimized using best practices.

Follow-up Questions: How would you optimize Supabase as the user base grows by 100x?

Q147. Explain the role of PostgreSQL (Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins) in a production system like SplitDude.

Expected Answer: In SplitDude, PostgreSQL is critical because it ensures robust handling of Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging PostgreSQL mitigates common failure cases. For example, Foreign Keys, Constraints, B-Tree Indexes, ACID Transactions, SQL Joins is optimized using best practices.

Follow-up Questions: How would you optimize PostgreSQL as the user base grows by 100x?

Q148. Explain the role of AWS (Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs) in a production system like SplitDude.

Expected Answer: In SplitDude, AWS is critical because it ensures robust handling of Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging AWS mitigates common failure cases. For example, Lambda Layers, API Gateway CORS, S3 Lifecycle Rules, SNS Topics, CloudWatch logs is optimized using best practices.

Follow-up Questions: How would you optimize AWS as the user base grows by 100x?

Q149. Explain the role of System Design (Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling) in a production system like SplitDude.

Expected Answer: In SplitDude, System Design is critical because it ensures robust handling of Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging System Design mitigates common failure cases. For example, Scalability, Connection Pooling, Edge Caching, Load Balancing, Write-heavy scaling is optimized using best practices.

Follow-up Questions: How would you optimize System Design as the user base grows by 100x?

Q150. Explain the role of Security (JWT verification, Cookie Security, CSRF Protection, Input Sanitization) in a production system like SplitDude.

Expected Answer: In SplitDude, Security is critical because it ensures robust handling of JWT verification, Cookie Security, CSRF Protection, Input Sanitization. This is handled using structured configurations and practices.

***Detailed Explanation:** Detailed architectural review indicates that leveraging Security mitigates common failure cases. For example, JWT verification, Cookie Security, CSRF Protection, Input Sanitization is optimized using best practices.*

Follow-up Questions: How would you optimize Security as the user base grows by 100x?

Q151. Explain the role of Performance (Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing) in a production system like SplitDude.

Expected Answer: In SplitDude, Performance is critical because it ensures robust handling of Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Performance mitigates common failure cases. For example, Lazy Loading, Shimmer skeletons, Code splitting, Bundle sizing is optimized using best practices.

Follow-up Questions: How would you optimize Performance as the user base grows by 100x?

Q152. Explain the role of Git/GitHub (Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts) in a production system like SplitDude.

Expected Answer: In SplitDude, Git/GitHub is critical because it ensures robust handling of Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Git/GitHub mitigates common failure cases. For example, Semantic Versioning, Conventional Commits, Pull Request Reviews, Merge Conflicts is optimized using best practices.

Follow-up Questions: How would you optimize Git/GitHub as the user base grows by 100x?

Q153. Explain the role of Behavioral (Conflict resolution, Handling requirements changes, Debugging under pressure) in a production system like SplitDude.

Expected Answer: In SplitDude, Behavioral is critical because it ensures robust handling of Conflict resolution, Handling requirements changes, Debugging under pressure. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Behavioral mitigates common failure cases. For example, Conflict resolution, Handling requirements changes, Debugging under pressure is optimized using best practices.

Follow-up Questions: How would you optimize Behavioral as the user base grows by 100x?

Q154. Explain the role of React (Hooks, State, Render Lifecycle, Virtual DOM, Memoization) in a production system like SplitDude.

Expected Answer: In SplitDude, React is critical because it ensures robust handling of Hooks, State, Render Lifecycle, Virtual DOM, Memoization. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging React mitigates common failure cases. For example, Hooks, State, Render Lifecycle, Virtual DOM, Memoization is optimized using best practices.

Follow-up Questions: How would you optimize React as the user base grows by 100x?

Q155. Explain the role of Next.js (App Router, Server Actions, Middleware, Dynamic Imports, Static Generation) in a production system like SplitDude.

Expected Answer: In SplitDude, Next.js is critical because it ensures robust handling of App Router, Server Actions, Middleware, Dynamic Imports, Static Generation. This is handled using structured configurations and practices.

Detailed Explanation: Detailed architectural review indicates that leveraging Next.js mitigates common failure cases. For example, App Router, Server Actions, Middleware, Dynamic Imports, Static Generation is optimized using best practices.

Follow-up Questions: How would you optimize Next.js as the user base grows by 100x?

End of Q&A; section.

Chapter 14: Possible Cross Questions

14.1 Why Lambda instead of EC2?

EC2 requires paying for idle CPU time, patching operating systems, and setting up complex auto-scaling groups. Lambda runs code only when triggered, scales infinitely out-of-the-box, and costs nothing when not in use.

14.2 Why SNS instead of SQS?

SNS is a publish-subscribe notification service that immediately pushes messages to subscribers. SQS is a queueing service that requires polling. For real-time updates and push alerts, SNS is preferred.

14.3 Why Supabase instead of Firebase?

Supabase utilizes standard PostgreSQL, offering relational integrity, foreign keys, transaction rollbacks, and native Row-Level Security. Firebase uses a NoSQL document database, which is poor for transactional ledgers.

14.4 Why Textract instead of Tesseract OCR?

Amazon Textract is a fully managed cognitive service that uses machine learning to identify vendor names and billing totals without requiring regex training. Tesseract is a raw character OCR that requires custom parsing pipelines.

Chapter 15: Project Demo Walkthrough

15.1 Step-by-Step Walkthrough

1. Landing Page: Start by showing the premium dark-themed landing page. Highlight that authentication uses secure Supabase Auth. 2. Unique Code: Show that signing up generates a unique Split ID (`unique_code`) which serves as an anonymous identifier. 3. Adding Friends: Search a friend's name and demonstrate that filtering happens in-memory instantly. Search by ID to demonstrate database fetching. 4. Creating a Group: Create a group with selected members. 5. Receipt OCR: Upload a receipt image, show the loading skeletal loader, and demonstrate that the merchant, amount, and date fields are auto-filled via AWS Textract. 6. Settlement: Record a settlement payment, and verify that the ledger balances update immediately.

Chapter 16: Resume Pitch Scripts

16.1 The 2-Minute Pitch

SplitDude is a serverless expense collaboration platform built around user privacy and system scalability. I designed the database schema on Supabase PostgreSQL with strict Row-Level Security. I integrated an AWS receipt OCR pipeline utilizing Amazon Textract, API Gateway, and Python Lambda functions. This allows users to scan receipts and pre-fill expense forms in seconds. Additionally, I implemented a greedy debt simplification algorithm to reduce transaction counts, and leveraged React Query for optimized edge caching.

16.2 The 5-Minute Pitch

Start with the 2-minute pitch, then deep dive into the AWS receipt pipeline: explaining how base64 payloads flow through Next.js proxy route to API Gateway, Lambda, S3, and Textract. Detail the 1-day lifecycle expiration rule for privacy. Then, explain Postgres RLS policies and how PgBouncer is used for connection pooling.

16.3 The 10-Minute Pitch

Extend the 5-minute pitch to explain the greedy debt simplification algorithm: how balances are sorted into creditor/debtor heaps and resolved in N-1 transactions. Walk through React Query caches and dynamic invalidation patterns to prevent stale UI state.

Chapter 17: Common Mistakes to Avoid

17.1 Technical Pitfalls

- Underestimating Postgres RLS: Never claim RLS is a frontend feature. Always emphasize it is enforced at the database engine level.
- Hardcoding API Gateway URL: Make sure to explain that Next.js proxy route secures the API Gateway URL from client exposure.
- Static cache: Avoid stating that UI updates instantly without query invalidation hooks.

17.2 Behavioral Anti-patterns

- Saying "I used libraries for everything": Emphasize that you designed the database triggers, IAM policies, and implemented the debt simplification algorithm from scratch.

Chapter 18: FAANG Level Architecture Assessment

18.1 Strengths

- Cloud-native serverless design makes it extremely cheap to host and auto-scales from zero.
- Strong relational database design ensures transaction safety and ledger accuracy.

18.2 Weaknesses & Scale Limits

- Large images can cause API Gateway payload size limits (10MB) to trigger error. Solutions include direct pre-signed S3 URL uploads.
- High write concurrency on Postgres can degrade connection performance if connection pooling is not configured.

Chapter 19: Quick Revision Cheat Sheets

19.1 Cheat Sheet

- Technology Stack: Next.js 16, React 19, TypeScript, Tailwind, React Query, Supabase, PostgreSQL, AWS Lambda, Textract, S3, API Gateway.
- Security: JWT tokens in HTTP-only cookies, Postgres Row-Level Security, API Gateway CORS, S3 lifecycle auto-delete.
- Optimization: React Query caching, local in-memory autocomplete filter, Recharts lazy loading, PgBouncer pooling.
- Core Logic: Greedy heap-based debt simplification resolving in N-1 transactions.